Lesson 9 Teacher's Key: On the Scoreboard

Unreal Engine (optional advanced track) - Instructor Reference Only

VIDEO GAME CREATION

What This Lesson Teaches

The UMG (Unreal Motion Graphics) Widget Blueprint system - Unreal's actual, standard on-screen UI framework, introduced properly this week after Lessons 6-8 deliberately deferred to simple Print String messages.

Level 1 - Full Worked Solution

- WBP_HUD is a Widget Blueprint - a separate, reusable UI asset containing Text Blocks, fundamentally different from the one-off Print String calls used in Lessons 6 and 8.
- UpdateHUD is a push-based Custom Event on the widget itself, called from BP_GameManager - the same placed Actor that already owns EnemiesRemaining and Lives - every time Score or Lives actually changes. Score, HUDWidget, and the Create Widget/Add to Viewport setup all live on BP_GameManager too, not the Level Blueprint, the same one-dedicated-manager pattern used since Lesson 6.
- Create Widget + Add to Viewport (done once, in BP_GameManager's BeginPlay) is the standard way to actually put a Widget Blueprint on screen - without this step, WBP_HUD would exist as an asset but never actually display.

Highlighted lines are new this week; everything else is code you already have.

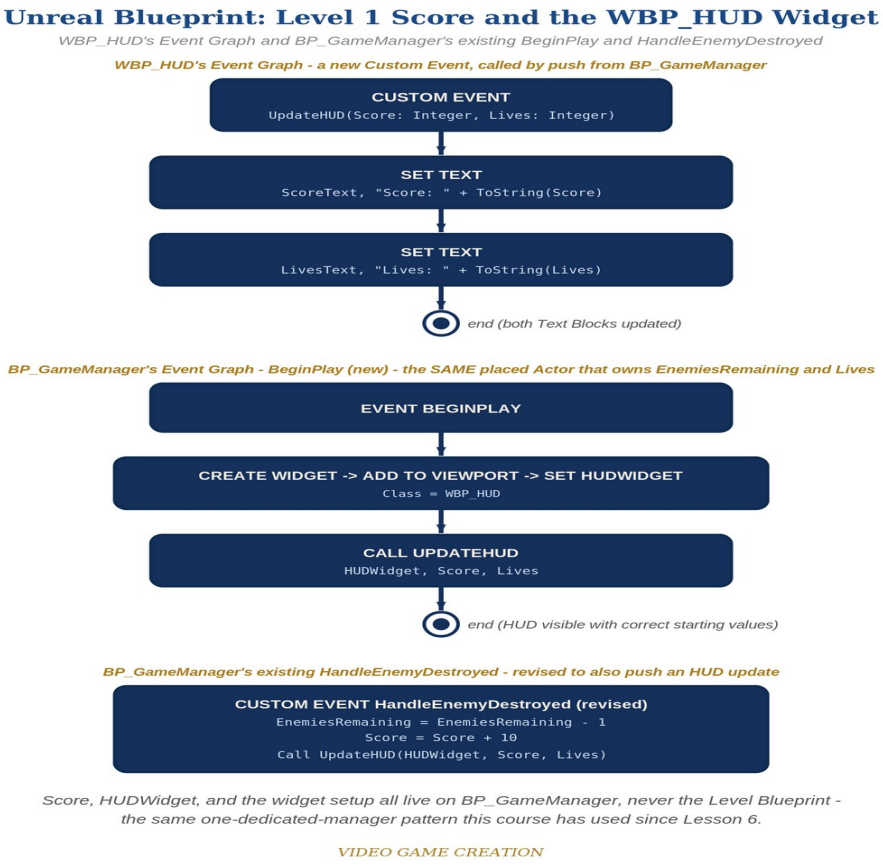
```
WBP_HUD's Event Graph (new):
Custom Event UpdateHUD(Score: Integer, Lives: Integer) ->
  Set Text(ScoreText, "Score: " + ToString(Score))
  Set Text(LivesText, "Lives: " + ToString(Lives))

BP_GameManager:
Event BeginPlay (new) ->
  Create Widget(Class = WBP_HUD) -> Add to Viewport -> Set HUDWidget
  Call UpdateHUD(HUDWidget, Score, Lives)

Custom Event HandleEnemyDestroyed ->
  EnemiesRemaining = EnemiesRemaining - 1
  Score = Score + 10
  Call UpdateHUD(HUDWidget, Score, Lives)
  Branch(EnemiesRemaining <= 0) -> True: Print String("You Win!", Print to
Screen = true)

Custom Event HandleEnemyEscaped ->
  Lives = Lives - 1
  Call UpdateHUD(HUDWidget, Score, Lives)
```

```
  Branch(Lives <= 0) -> True: Print String("Game Over", Print to Screen =
true)
```



WBP_HUD's Event Graph and BP_GameManager's BeginPlay and HandleEnemyDestroyed - the node-for-node equivalent of this solution.

Why This Structure

- This mirrors Godot's push-based update_hud() function almost exactly, just expressed through Unreal's widget/dispatcher idiom instead of GDScript - worth naming this cross-engine parallel explicitly for students working in both. Godot's version lives on GameManager; Unreal's lives on BP_GameManager - the same architectural choice, two engines.
- Keeping ScoreText/LivesText updates entirely inside UpdateHUD (never set directly from BP_GameManager) mirrors the same discipline established in Godot: one function is the sole owner of what the display shows.

Level 2 / Level 3 Talking Points

- For students who also did Unity this week: contrast Unreal's push-based UpdateHUD (called only when something changes) against Unity's poll-based HUD.cs Update() (checked every frame) - both are legitimate, and this is a good moment to ask which one they'd choose for a hypothetical HUD with many rapidly-changing values.
- This is the most setup-heavy engine this week (Widget Blueprint creation, Designer view layout, Create Widget/Add to Viewport wiring) - worth acknowledging directly with

students working in Unreal that more scaffolding this week is expected, not a sign they're doing something wrong.

Level 2 - complete code:

```
// Level 2 is verification only - the Level 1 code is already correct, so
no lines change.
// Checks: 3 kills read exactly "Score: 30"; an escape moves lives but
never score;
// both numbers are correct the instant the game starts, not only after
the first kill or escape.
```

Level 3 - complete code (all goals combined):

Highlighted lines are the Level 3 additions on top of the Level 1 code.

```
BP_GameManager - Level 3:
Custom Event HandleEnemyDestroyed ->
  EnemiesRemaining = EnemiesRemaining - 1
  Score = Score + Random Integer in Range(5, 15)
  Call UpdateHUD(HUDWidget, Score, Lives)
  Branch(EnemiesRemaining <= 0) -> True: Print String("You Win!", Print to
Screen = true)

WBP_HUD Designer view: set font size and color on ScoreText and LivesText
for the styled HUD.
```

Common Student Mistakes

- Forgetting Add to Viewport after Create Widget - the widget exists in memory but is never actually shown on screen.
- Calling UpdateHUD before HUDWidget is actually set (wiring order matters - Create Widget/Add to Viewport/Set HUDWidget must all happen before the first UpdateHUD call).
- Text Block name mismatches between the Designer view and the Set Text nodes in the Event Graph - these must reference the exact same Text Block.